

# CSCI 5551 Introduction to Robotics

“Surveillance Using Intelligent Agents: Multi-Camera Visual Tracking of Mobile Robots.”

Adam Meyers, Max LaLonde, Alexandra Postolaki

([meye3224@umn.edu](mailto:meye3224@umn.edu), [lalon040@umn.edu](mailto:lalon040@umn.edu), [post022@umn.edu](mailto:post022@umn.edu))

## ABSTRACT

A multi-camera surveillance system was developed to seamlessly track a mobile agent throughout a complex 3D environment. A Unity simulation was created, in which a mobile agent traveled random paths, while camera units dispersed throughout the environment identified and tracked the mobile agent, using the YOLOv8 algorithm. Each camera agent communicates with a central control system, coordinating tracking between camera units, allowing seamless transition of primary tracking responsibilities between nearby camera units. This system proved functional in certain conditions, but had many limitations due to lack of time and data for further training of the object detection model, and a need for further optimizations to the central control system. With these additional improvements, this system may prove useful in real-world surveillance applications, where targets may need to be tracked throughout a complex environment and across multiple fields of view.

## PROBLEM DESCRIPTION

### 1. Introduction

Modern surveillance systems are increasingly relying on automated networks of intelligent agents (cameras) to monitor large and complex environments. As it is rare for a security system to be able to cover an entire area with only one camera, there arises a need to continuously track targets between the fields of view of multiple cameras [2]. Without proper communication and coordination between agents, targets can get lost as they move beyond a specific camera's line of sight.

The goal of this project is to design and simulate a multi-camera visual tracking system in which multiple intelligent agents cooperate to monitor a complex 3D environment. Each agent will control a camera capable of panning to track the movement of a mobile agent moving throughout the environment. When an agent detects a target, it will communicate with all other agents within the network, preparing nearby agents to overtake the responsibility of tracking the target when it appears within their field of vision.

### 2. Previous Literature

Cooperative agents for tracking have various applications ranging from improving surveillance systems for search operations [11] to monitoring people [18]. There is a variety of previous literature surrounding multi-camera visual tracking systems. Notably, [3], [6], and [12] describe

a set of various approaches for multi-sensor object tracking. Papers [2] and [4] describe a special case of multi-sensor cooperation systems where both the sensing and tracked objects are capable of movement. This is less directly applicable to this project, but may have some utility as the camera units in this project do have a small range of motion. [13] highlights a global data-association method that shows how one can combine detections from cameras, and handle occlusions, to maintain a continuous track of multiple objects.

[5] discusses a potential method to allow cameras with a partial view of an object to still track the object, via measuring the shape similarity with a camera that has full view of the object. Similarly, it could be possible to implement single camera algorithms to identify partial views, or even estimate position after completely leaving a camera's field of view [7], [8], [14]. Methods proposed by [16] and [17] for continuous learning, adapting, and good feature selection can help improve accuracy in tracking. This could be used to enhance the ability for cameras to tradeoff primary tracking responsibilities, though may be beyond the scope of this project.

[20] Presents a possible future extension for this project should the work be carried further in the future. Right now our simulation is equipped to track only one robot however [20] presents a method of tracking multiple robots in an environment using a "density-based spatial clustering of applications with noise to detect the robots and a single k-dimensional tree to keep track of them as they move in an indoor arena."

### 3. Simulation

The simulation is built in the Unity game engine where a pre-existing scene and robot model are utilized as the bases of the project to focus on the tracking system rather than creating new assets from scratch. To replicate real-world surveillance scenarios, similar to the approach in [1], the simulation is designed to include nodes and pathways for the robot to follow semi-randomly throughout the scene. The mobile robot is programmed with the ability to stop and rotate at various pathway points for realistic movements. In addition to the mobile robot, there are various intelligent camera units throughout the scene that work together to observe the robot as it moves throughout the scene. This intelligent tracking behavior is what is explored throughout the rest of this report.

### 4. Entities

At the core of the simulation are 5 distinct entities: the scene, the nodes, the mobile robot, the camera units, and the central camera control.

- i. Scene: A pre-built scene of an urban, city environment was chosen for the simulation. This emulates a common environment in which surveillance may be necessary in the real world. The many tall buildings and side streets also ensure that a network of communicating camera agents is necessary to allow continuous tracking of a mobile agent.

ii. Nodes: As mentioned before, nodes are points dispersed throughout the scene that are linked together to form pathways that the mobile robot is programmed to follow. These nodes were manually placed with connecting pathways following sidewalks and streets within the city environment, allowing for realistic navigation paths throughout the environment.

iii. Mobile robot: The mobile robot uses a pre-built model and is tasked to continuously follow the web of nodes and pathways as the target object in the simulation. At each node, the robot randomly chooses an adjacent connecting node to travel to, pausing and rotating when necessary so that it always faces the direction of movement. The robot will avoid selecting the node where it came from, unless it is the only available path. This random movement path allows for a wide range of largely unpredictable navigation paths for the cameras to keep track of.

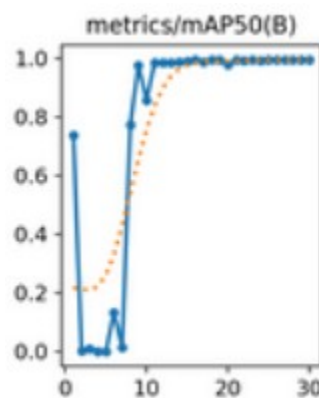
iv. Intelligent camera agents: Multiple camera units are placed throughout the map. These entities work together to track the location of the target robot. Each camera is equipped with 2 revolute joints allowing vertical and horizontal panning. As the robot moves into the field of view of one camera, that camera recognizes and tracks the robot until the robot moves out of the camera's reachable field of view. Each camera also communicates with a central camera control when it is tracking a target.

v. Camera control: Communication topology, the arrangement of how the information is exchanged between agents of this system, plays an important role in information flow for how the cameras work together [15], [19], [21]. A central control system coordinates cooperation between all camera units. This emulates the likely real-world situation of having a single physical computer overseeing the camera agents. This control system performs complex tasks that may be too computationally expensive to run on a single camera's internal logic, as well as allow communication between camera agents. Notably, when a camera is actively tracking the mobile agent, the central control will notify all nearby camera units to prepare them for a seamless tradeoff when sight is lost.

## 5. Model Training, Object Detection, and Tracking

For model training, object detection and tracking of the target robot, Ultralytics YOLO [9] is utilized to train a model for object detection. The in-camera detection is repeated for all the intelligent agents within the simulator. Specifically, due to its performance in terms of accuracy and speed, YOLOv8 is an ideal choice for simulations as it is a deep learning model designed for object detection in real time [10]. In this particular simulation, Robotflow was used to create a dataset of 496 images followed by code provided by Ultralytics to train the YOLOv8 model on our custom data set with a split of 70%-20%-10% on which images are used for training the model for this specific task, which are used in validation to prevent overfitting and monitor performance, and which are used for testing after training to evaluate the final model, respectively. One important note is that the total dataset of 496 images includes 20 snapshots in which the robot isn't present so that the model has an idea of what a 'null' shot could look like. The dataset includes augmented images where they were darkened, brightened, rotated, sheared,

and included noise in order to increase accuracy in example cases where the pictures captured from the cameras could have been taken in darker areas, brighter areas, or if the cameras were slightly rotated producing an overall rotated shot of the robot. Roboflow calculates the mAP at 50, which is equal to the mean of the Average Precision across all of the classes in a model at a 50% IoU threshold. This was found to be 99.5% which is good considering we want to have high accuracy of a model when identifying and classifying objects within an image. The precision of accuracy of the positive predictions was 99.8% and the recall, the measurement of what labels were successfully identified, was 100%. When training the YOLOv8 model on the custom dataset for 30 epochs, we reached an approximately similar trend for a mAP of a 50% threshold (also reaching 99.5%) as seen in Figure 1A below.



**Figure 1A: mAP50 for AttackBot dataset**

## RESULTS AND INSIGHTS

We created a simulation in which a trained YOLOv8 model, trained specifically on the model robot, is used to track the mobile robot from different cameras. At every frame, the camera which successfully detects the robot is determined. The “main” camera is selected based on whichever camera currently has valid detection. When a camera detects the robot, the system switches the active view to that camera.

The YOLOv8 Python script is what performs the detection using the best-performing trained weights. For each frame, it outputs a JSON file which contains the identifying number of the camera which detected the robot and bounding box information for the robot. This JSON file is then read inside Unity and the data is used to sync with the simulation in order to determine which camera is currently tracking the robot and visualize what each camera sees.

Here, a known limitation that we’ve noticed is that switching the active view to the detecting camera breaks if two cameras happen to detect the robot simultaneously. Another limitation had to do with the images captured by the cameras. They appeared darker than what is perceived when viewing the simulation. And although some augmentations were added, such as the addition of noise or darkening of images in the dataset, the cameras still had occasional moments when they were unable to detect the robot when it was too far away or about 50% occluded.

Multi-camera tracking in our system proved to be much more difficult than simply adding more cameras. The real challenge is coordinating which camera is considered the active “tracking”, or “detecting”, camera. The accuracy of YOLOv8 was also strongly influenced by lighting conditions, the degree of which the robot was partially occluded, and also the distance of the mobile robot to the cameras. We’ve also observed subtle delays in the pipeline since Unity saves the frames, Python performs YOLO detection and creates the JSON files, and Unity then reloads those results. This highlights the realization that real-time robotics pipelines need careful synchronization to help with smooth, real-time tracking performance.

## **LESSONS LEARNT**

One lesson we learned was the importance of planning designs ahead of time. There were multiple points throughout the process where a lot of time was spent trying to find a Unity package or some other tool that would allow us to do something, or doing work that ended up being unnecessary for the final product. If we had taken some additional time to plan the exact implementation ahead of time, these issues could have been largely avoided. Even with the additional planning overhead this would have added, it still likely could have been beneficial to hold some sort of planning meeting at the beginning, rather than just starting with an end goal and a few basic milestones.

Another lesson was that we should expect additional time to be needed for learning new softwares and languages. Getting Unity properly set up on all 3 personal computers ended up being a far greater struggle than initially anticipated, and even after it was set up, there was still a notable amount of learning pains when it came to discovering how to do certain things within the Unity editor. It was still incredibly interesting to learn these new systems and techniques, but it added a lot of additional work to our plate that we had not expected. This was likely an unavoidable situation, as the alternatives to Unity were also very unfamiliar. However, in future projects, if we are ever required to use a program or interface that is unfamiliar to us, we will be sure to allot more time for learning the basics of such a system.

Additionally, we likely could have benefited from starting model training earlier along in the process. Training models on large quantities of data takes a long time, so we weren’t able to have as many iterations of the model as we may have liked. This proved problematic for the final product, as while the model performed great on the testing data we provided, when actually deployed within the simulation it was observed to occasionally fail in certain situations, such as when the mobile agent was very far from the camera.

## **FUTURE WORK AND PROBLEMS ENCOUNTERED**

A few different problems were encountered over the course of work. One such problem was difficulty in learning how to do certain tasks within the Unity engine, most notably with the object detection algorithm. While it is seemingly possible to get the YOLOv8 model working within Unity, we continued to run into issues with getting the trained model file to be recognized within Unity’s scripting engine, and instead opted to use a python script to do the image analysis, communicating with Unity via PNG output from the cameras and JSON output from the python

script. In the future, this should be amended to work within a singular system, either entirely in Unity, ROS, or another scripting language.

Another one of the primary problems encountered was getting a single computer to run an object detection model for an entire network of camera units. In a physical implementation of this project, each individual camera would likely be doing a small portion of the computation, allowing for a greater number of images to be processed. This is a core reason for the split between camera agents and a central control system. However, in the simulation all computations must be done by a single computer, which led to us having to limit the speed at which images from the cameras were analyzed.

Additional work for the future includes the implementation of more robust camera communication abilities, ideally being able to identify not just nearby cameras, but specifically cameras that would next have line of sight, which might not necessarily be nearby to the current tracking camera. In addition, taking into account the direction of movement to predict which nearby camera would next see the mobile agent would also be a significant improvement to reduce wasted computation on cameras in the opposite direction that the mobile agent is moving. Both of these additions were beyond the scope of this project, but would make for incredibly impactful improvements.

Additionally, the model is currently only trained to identify one specific mobile agent. If the camera system was ever needed to track something that looked different than this agent, it would be completely unable to do so. This is obviously not ideal for an actual surveillance use case, where targets may need to frequently change. A necessary improvement would be the implementation of models with greater identification abilities, being able to identify multiple different faces, car models, and other types of mobile objects that people may want to track with surveillance cameras.

The final notable piece of future work is a physical implementation of the camera system. This would likely be implemented with CMOS cameras due to their lower cost, power consumption, and processing speeds. A physical implementation would likely also run into additional complications that could not be simulated on a computer, with additional variables such as weather, wild animals, and hardware degradation. Therefore, it's important to run additional testing on a physical model even if everything functions well in computer simulations.

## **TASK DISTRIBUTION**

For this project, Adam did the setup for the simulation which includes finding the Unity map of a city, creating a mobile robot that moves along pathways generated by nodes, and making cameras that took shots of the map while also displaying the camera frames on screen when it detected the robot in its field of view. To connect the python script that did the object detection to the simulator on Unity, Adam was the one who wrote code to create JSON files that was the overall communication between the simulator to know which camera is tracking the robot and the detection being done in python with the images from Unity.

In addition to Adam's work with making the mobile robot move, Alexandra added realistic turning of the robot and implemented YOLO detection with YOLOv8, created and labeled the custom datasets with proper augmentations used in the simulation to detect the robot, and also trained the YOLOv8 model with the datasets. Apart from that, Alexandra also generated metrics to determine the accuracy of the trained YOLO model.

Max aided in brainstorming ideas for the project as well as creating the presentation slides and writing up the report with the research that was conducted. Although Adam and Alexandra wrote parts of the report as well, it was also Max who aided in peer reviewing and bringing up any concerns of where our report may not have met the expected requirements for the final report.

Overall, we worked collaboratively with bringing new ideas on how to improve our simulation. In areas where one group member felt more confident in, that's where they took the lead. We learned from one another and built off of each other which was what made this project relatively successful.

## **CONCLUSION**

There has been a large variety of proposed solutions to the surveillance system problem, many of which are described at the start of this paper. We successfully showed one such solution, using a central camera control that coordinates multiple cameras to track a mobile agent, using the YOLOv8 object detection model. This is a sufficient solution but does still have its limitations. Coordinating camera hand-offs is quite difficult and determining which camera should be treated as the "active" camera is not as straightforward as checking which camera is currently detecting an object. More development needs to be done on the logic for this issue.

The computational struggles we encountered highlight the need for a multi-camera distributed system. Our Unity based simulation was unfortunately too taxing on our own computers, as a single computer needed to run multiple instances of the YOLOv8 model, and manage multiple cameras. In a physical implementation, this issue could be resolved by running some computation within individual camera units, in addition to the central control unit. In addition, a lack of time and data for training the object detection model resulted in notable limitations, such as occasional failure to detect the mobile agent, and being limited to identifying just one type of object. However, this system has shown notable potential, successfully tracking the mobile agent in certain conditions, and with additional time and work, could potentially result in a robust and accurate surveillance system.

## **REFERENCES**

[1] W. Luo, P. Sun, F. Zhong, W. Liu, T. Zhang and Y. Wang, "End-to-End Active Object Tracking and Its Real-World Deployment via Reinforcement Learning," in IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 42, no. 6, pp. 1317-1332, 1 June 2020, doi: 10.1109/TPAMI.2019.2899570.

keywords: {Object tracking;Cameras;Target tracking;Reinforcement learning;Robot vision systems;Active object tracking;reinforcement learning;environment augmentation},

[2] A. Khan, B. Rinner and A. Cavallaro, "Cooperative Robots to Observe Moving Targets: Review," in IEEE Transactions on Cybernetics, vol. 48, no. 1, pp. 187-198, Jan. 2018, doi: 10.1109/TCYB.2016.2628161.

keywords: {Robot kinematics;Robot sensing systems;Mobile robots;Planning;Target tracking;Cooperative mobile robots;cooperative observation;motion planning;multirobot system;surveillance},

[3] R. T. Collins, A. J. Lipton, H. Fujiyoshi and T. Kanade, "Algorithms for cooperative multisensor surveillance," in Proceedings of the IEEE, vol. 89, no. 10, pp. 1456-1477, Oct. 2001, doi: 10.1109/5.959341.

keywords: {Video surveillance;Monitoring;Humans;Sensor systems;Vehicle detection;Control system synthesis;Automatic control;Graphical user interfaces;Information security;Real time systems},

[4] S. Minaeian, J. Liu and Y. -J. Son, "Vision-Based Target Detection and Localization via a Team of Cooperative UAV and UGVs," in IEEE Transactions on Systems, Man, and Cybernetics: Systems, vol. 46, no. 7, pp. 1005-1016, July 2016, doi: 10.1109/TSMC.2015.2491878.

keywords: {Cameras;Surveillance;Vehicles;Motion detection;Optical sensors;Real-time systems;Algorithms;automation cooperative systems;geographic information systems (GISs);position measurement;Algorithms;automation cooperative systems;geographic information systems (GISs);position measurement},

[5] S. Belongie, J. Malik and J. Puzicha, "Shape matching and object recognition using shape contexts," in IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 24, no. 4, pp. 509-522, April 2002, doi: 10.1109/34.993558.

keywords: {Shape;Object recognition},

[6] J. Black, T. Ellis and P. Rosin, "Multi view image surveillance and tracking," Workshop on Motion and Video Computing, 2002. Proceedings., Orlando, FL, USA, 2002, pp. 169-174, doi: 10.1109/MOTION.2002.1182230. keywords: {Surveillance;Cameras;Object

detection;Trajectory;Computer science;Video sequences;Robustness;Layout;Uncertainty;Intelligent networks},

[7] A. Adam, E. Rivlin and I. Shimshoni, "Robust Fragments-based Tracking using the Integral Histogram," 2006 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR'06), New York, NY, USA, 2006, pp. 798-805, doi: 10.1109/CVPR.2006.256.

keywords: {Robustness;Histograms;Voting;Target tracking;Video sequences;Torso;Humans;Statistics;Data structures;Data mining},

[8] H. Grabner, J. Matas, L. Van Gool and P. Cattin, "Tracking the invisible: Learning where the object might be," 2010 IEEE Computer Society Conference on Computer Vision and Pattern Recognition, San Francisco, CA, USA, 2010, pp. 1285-1292, doi: 10.1109/CVPR.2010.5539819.



keywords: {Target tracking;Object detection;Medical services;Biomedical equipment;Face detection;Context modeling;Voting;Computer vision;Laboratories;Biomedical imaging},

[9] J. Redmon, S. Divvala, R. Girshick and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, USA, 2016, pp. 779-788, doi: 10.1109/CVPR.2016.91.

keywords: {Computer architecture;Microprocessors;Object detection;Training;Real-time systems;Neural networks;Pipelines},

[10] R. Varghese and S. M., "YOLOv8: A Novel Object Detection Algorithm with Enhanced Performance and Robustness," *2024 International Conference on Advances in Data Engineering and Intelligent Computing Systems (ADICS)*, Chennai, India, 2024, pp. 1-6, doi: 10.1109/ADICS58448.2024.10533619.

keywords: {YOLO;Performance evaluation;Technological innovation;Computer vision;Heuristic algorithms;Speech recognition;Benchmark testing;YOLOv8;Object Detection;Performance Enhancement;Robustness;Computational Efficiency;Computer Vision Systems},

[11] T. Furukawa, F. Bourgault, B. Lavis and H. F. Durrant-Whyte, "Recursive Bayesian search-and-tracking using coordinated uavs for lost targets," *Proceedings 2006 IEEE International Conference on Robotics and Automation, 2006. ICRA 2006.*, Orlando, FL, USA, 2006, pp. 2521-2526, doi: 10.1109/ROBOT.2006.1642081. keywords: {Bayesian methods;Unmanned aerial vehicles;Target tracking;Remotely operated vehicles;Mobile robots;Australia;Filtering;Marine vehicles;Robot kinematics;Switches},

[12] T. Adachi, N. Hayashi and S. Takai, "Cooperative Target Tracking by Multiagent Camera Sensor Networks via Gaussian Process," in *IEEE Access*, vol. 10, pp. 71717-71727, 2022, doi: 10.1109/ACCESS.2022.3188857. keywords: {Cameras;Target tracking;Uncertainty;Gaussian processes;Mathematical models;Numerical models;Estimation;Multiagent system;camera sensor network;Gaussian process},

[13] Li Zhang, Yuan Li and R. Nevatia, "Global data association for multi-object tracking using network flows," *2008 IEEE Conference on Computer Vision and Pattern Recognition*, Anchorage, AK, USA, 2008, pp. 1-8, doi: 10.1109/CVPR.2008.4587584. keywords: {Object detection;Trajectory;Optimization methods;Iterative algorithms;Intelligent robots;Iterative methods;Event detection;Costs;Linear programming;Intelligent systems},

[14] O. Javed, K. Shafique and M. Shah, "Appearance modeling for tracking in multiple non-overlapping cameras," *2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR'05)*, San Diego, CA, USA, 2005, pp. 26-33 vol. 2, doi: 10.1109/CVPR.2005.71. keywords: {Cameras;Brightness;Computer vision;Transfer functions;Lighting;Layout;Material properties;Training data;Trajectory;Surveillance},

[15] R. Olfati-Saber, J. A. Fax and R. M. Murray, "Consensus and Cooperation in Networked Multi-Agent Systems," in *Proceedings of the IEEE*, vol. 95, no. 1, pp. 215-233, Jan. 2007, doi:

10.1109/JPROC.2006.887293. keywords: {Multiagent systems;Performance analysis;Sensor fusion;Failure analysis;Information analysis;Algorithm design and analysis;Robustness;Network topology;Convergence;Matrices;Consensus algorithms;cooperative control;flocking;graph Laplacians;information fusion;multi-agent systems;networked control systems;synchronization of coupled oscillators},

[16] Z. Kalal, K. Mikolajczyk and J. Matas, "Tracking-Learning-Detection," in *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 34, no. 7, pp. 1409-1422, July 2012, doi: 10.1109/TPAMI.2011.239.

keywords: {Training;Detectors;Target tracking;Streaming media;Real time systems;Estimation;Long-term tracking;learning from video;bootstrapping;real time;semi-supervised learning.},

[17] Jianbo Shi and Tomasi, "Good features to track," *1994 Proceedings of IEEE Conference on Computer Vision and Pattern Recognition*, Seattle, WA, USA, 1994, pp. 593-600, doi: 10.1109/CVPR.1994.323794.

keywords: {Tracking;Machine vision;Feature extraction},

[18] F. Fleuret, J. Berclaz, R. Lengagne and P. Fua, "Multicamera People Tracking with a Probabilistic Occupancy Map," in *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 30, no. 2, pp. 267-282, Feb. 2008, doi: 10.1109/TPAMI.2007.1174. keywords: {Dynamic programming;Robustness;Streaming media;Trajectory;Hidden Markov models;Head;Video surveillance;Cameras;Loss measurement;Humans;Multi-people tracking;Multi-camera;Visual surveillance;Probabilistic occupancy map;Dynamic Programming;Hidden Markov Model;Multi-people tracking;Multi-camera;Visual surveillance;Probabilistic occupancy map;Dynamic Programming;Hidden Markov Model},

[19] L. Moreau, "Stability of multiagent systems with time-dependent communication links," in *IEEE Transactions on Automatic Control*, vol. 50, no. 2, pp. 169-182, Feb. 2005, doi: 10.1109/TAC.2004.841888. keywords: {Multiagent systems;Convergence;Communication system control;Control systems;Oscillators;Stability analysis;Network topology;Physics;Equations;Distributed decision making;Multiagent systems;set-valued Lyapunov theory;stability analysis;swarms;synchronization},

[20] C. Iaboni, H. Patel, D. Lobo, J. -W. Choi and P. Abichandani, "Event Camera Based Real-Time Detection and Tracking of Indoor Ground Robots," in *IEEE Access*, vol. 9, pp. 166588-166602, 2021, doi: 10.1109/ACCESS.2021.3133533.

keywords: {Cameras;Robots;Robot vision systems;Tracking;Service robots;Real-time systems;Clustering algorithms;Event cameras;multi-robot systems;detection and tracking;clustering and pattern recognition},

[21] Robinson, L., Gadd, M., Newman, P., Martini, D.D. (2024). *Robot-Relay: Building-Wide, Calibration-Less Visual Servoing with Learned Sensor Handover Networks*. In: Ang Jr, M.H.,

Khatib, O. (eds) Experimental Robotics. ISER 2023. Springer Proceedings in Advanced Robotics, vol 30. Springer, Cham. [https://doi.org/10.1007/978-3-031-63596-0\\_12](https://doi.org/10.1007/978-3-031-63596-0_12)